

Menu-Driven C Program on Increasing Patterns using Nested Loops

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

Objective

To write a menu-driven C program that uses a switch statement to display five different 'increasing' star, number and alphabet patterns using nested loops, without using arrays. Each pattern is implemented as a separate, parameterless function with no return value (void). For every activity, the algorithm is designed first, a test-case table is prepared, and then the program is written, compiled, executed, and verified against the prepared test cases.

Tools Used

Operating System: Windows / Linux

Compiler: GCC (GNU Compiler Collection) / any
standard C compiler Editor / IDE: Code::Blocks, Dev-

```
#include <stdio.h>

/* ---- Function prototypes (parameterless, no return value) ---- */
void increasingTriangle(void); void
rowNumberTriangle(void); void
naturalNumberPattern(void); void
consecutiveNumberTriangle(void);
void alphabetPattern(void);

int main(void)
{
    int choice;

do
{
    printf("\n==== MENU: Increasing Patterns =====\n");
    printf("1. Increasing right-angled triangle\n");
    printf("2. Increasing row-number triangle\n");
    printf("3. Increasing natural-number pattern\n");
    printf("4. Consecutive-number triangle\n");
    printf("5. Increasing alphabet pattern\n");
    printf("6. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1: increasingTriangle();          break;
```

C++, VS Code, or online GDB

Complete C Program

The complete menu-driven program is given below. It displays a menu of five pattern choices plus an Exit option, reads the user's choice, and uses a switch statement to call the corresponding function. Each pattern function reads the number of rows n and prints its pattern using two nested for loops; no arrays are used anywhere in the program.

```

        case 2: rowNumberTriangle();          break;
case 3: naturalNumberPattern();              break;
case 4: consecutiveNumberTriangle();         break;
case 5: alphabetPattern();                   break;
case 6: printf("Exiting program.\n"); break;
        default: printf("Invalid choice! Please try again.\n");
    }
} while (choice != 6);

return 0;
}

/* 1. Increasing right-angled triangle */
void increasingTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
}

/* 2. Increasing row-number triangle */
void rowNumberTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", i);
        printf("\n");
    }
}

/* 3. Increasing natural-number pattern */
void naturalNumberPattern(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {

```

```

        for (j = 1; j <= i; j++)
printf("%d ", j);
        printf("\n");
    }
}

/* 4. Consecutive-number triangle */
void consecutiveNumberTriangle(void)
{
    int n, i, j, num = 1;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", num);
            num++;
        }
        printf("\n");
    }
}

/* 5. Increasing alphabet pattern (assume 1 <= n <= 26) */
void alphabetPattern(void)
{
    int n, i, j;
    char ch;
    printf("Enter number of rows (1-26): ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        ch = 'A';
        for (j = 1; j <= i; j++)
        {
            printf("%c ", ch);
            ch++;
        }
        printf("\n");
    }
}

```

Activity 1: Increasing Right-Angled Triangle

This program accepts the number of rows, n , from the user and prints a right-angled triangle of asterisks (*) in which row i contains i asterisks.

Algorithm

1. Start.
2. Call the function `increasingTriangle()`.
3. Input the number of rows, n , from the user.
4. Set outer loop counter $i = 1$; repeat while $i \leq n$, incrementing i by 1 each time.
5. Inside the outer loop, set inner loop counter $j = 1$; repeat while $j \leq i$, incrementing j by 1 each time.
6. In the inner loop, print an asterisk (*) followed by a space.
7. After the inner loop finishes, print a newline to move to the next row.
8. Repeat steps 5-7 until the outer loop condition ($i \leq n$) becomes false.
9. Return control to the main menu.
10. Stop.

Test Case Table

TC No.	Input (n)	Expected Output	Actual Output	Result
1	4	* * * * * * * * * *	* * * * * * * * * *	Pass
2	1	*	*	Pass
3	0	(no rows printed)	(no rows printed)	Pass

Sample Program Output (as displayed on screen)

```
===== MENU: Increasing Patterns =====
1.    Increasing right-angled triangle
2.    Increasing row-number triangle
3.    Increasing natural-number pattern
4.    Consecutive-number triangle
5.    Increasing alphabet pattern
6.    ExitEnter your choice: 1
Enter number of rows: 4
*
* *
* * *
* * * *
```

Activity 2: Increasing Row-Number Triangle

This program accepts the number of rows, n , and prints a triangle in which the row number i is repeated i times in row i .

Algorithm

1. Start.
2. Call the function `rowNumberTriangle()`.
3. Input the number of rows, n , from the user.
4. Set outer loop counter $i = 1$; repeat while $i \leq n$, incrementing i by 1 each time.
5. Inside the outer loop, set inner loop counter $j = 1$; repeat while $j \leq i$, incrementing j by 1 each time.
6. In the inner loop, print the value of i followed by a space (the row number is repeated).
7. After the inner loop finishes, print a newline to move to the next row.
8. Repeat steps 5-7 until the outer loop condition ($i \leq n$) becomes false.
9. Return control to the main menu.
10. Stop.

Test Case Table

TC No.	Input (n)	Expected Output	Actual Output	Result
1	4	1 2 2 3 3 3 4 4 4 4	1 2 2 3 3 3 4 4 4 4	Pass

2	1	1	1	Pass
3	0	(no rows printed)	(no rows printed)	Pass

Sample Program Output (as displayed on screen)

```

Enter your choice: 2
Enter number of rows: 4
1
2 2
3 3 3
4 4 4 4

```

Activity 3: Increasing Natural-Number Pattern

This program accepts the number of rows, n , and prints, in each row i , the natural numbers from 1 up to i .

Algorithm

1. Start.
2. Call the function `naturalNumberPattern()`.
3. Input the number of rows, n , from the user.
4. Set outer loop counter $i = 1$; repeat while $i \leq n$, incrementing i by 1 each time.
5. Inside the outer loop, set inner loop counter $j = 1$; repeat while $j \leq i$, incrementing j by 1 each time.
6. In the inner loop, print the value of j followed by a space.
7. After the inner loop finishes, print a newline to move to the next row.
8. Repeat steps 5-7 until the outer loop condition ($i \leq n$) becomes false.
9. Return control to the main menu.
10. Stop.

Test Case Table

TC No.	Input (n)	Expected Output	Actual Output	Result
1	4	1 1 2 1 2 3 1 2 3 4	1 1 2 1 2 3 1 2 3 4	Pass

2	1	1	1	Pass
3	0	(no rows printed)	(no rows printed)	Pass

Sample Program Output (as displayed on screen)

```

Enter your choice: 3
Enter number of rows: 4
1
1 2
1 2 3
1 2 3 4

```

Activity 4: Consecutive-Number Triangle

This program accepts the number of rows, n , and prints consecutive natural numbers (1, 2, 3, ...) arranged in a triangular pattern, continuing the count across rows.

Algorithm

1. Start.
2. Call the function consecutiveNumberTriangle().
3. Input the number of rows, n , from the user.
4. Initialise a counter, $num = 1$, before the outer loop begins.
5. Set outer loop counter $i = 1$; repeat while $i \leq n$, incrementing i by 1 each time.
6. Inside the outer loop, set inner loop counter $j = 1$; repeat while $j \leq i$, incrementing j by 1 each time.
7. In the inner loop, print the value of num followed by a space, then increment num by 1.
8. After the inner loop finishes, print a newline to move to the next row.
9. Repeat steps 6-8 until the outer loop condition ($i \leq n$) becomes false.
10. Return control to the main menu.
11. Stop.

Test Case Table

TC No.	Input (n)	Expected Output	Actual Output	Result
--------	-----------	-----------------	---------------	--------

1	4	1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10	Pass
2	1	1	1	Pass
3	0	(no rows printed)	(no rows printed)	Pass

Sample Program Output (as displayed on screen)

```

Enter your choice: 4
Enter number of rows: 4
1
2 3
4 5 6
7 8 9 10

```

Activity 5: Increasing Alphabet Pattern

This program accepts the number of rows, n ($1 \leq n \leq 26$), and prints uppercase English letters from A up to the letter corresponding to each row.

Algorithm

1. Start.
2. Call the function `alphabetPattern()`.
3. Input the number of rows, n , from the user (assume $1 \leq n \leq 26$).
4. Set outer loop counter $i = 1$; repeat while $i \leq n$, incrementing i by 1 each time.
5. Inside the outer loop, initialise a character variable $ch = 'A'$.
6. Set inner loop counter $j = 1$; repeat while $j \leq i$, incrementing j by 1 each time.
7. In the inner loop, print the character ch followed by a space, then increment ch to the next letter.
8. After the inner loop finishes, print a newline to move to the next row.
9. Repeat steps 5-8 until the outer loop condition ($i \leq n$) becomes false.
10. Return control to the main menu.
11. Stop.

Test Case Table

TC No.	Input (n)	Expected Output	Actual Output	Result
1	4	A A B A B C A B C D	A A B A B C A B C D	Pass
2	1	A	A	Pass
3	26	26-row pattern ending in full alphabet on row 26	26-row pattern ending in full alphabet on row 26	Pass

Sample Program Output (as displayed on screen)

```

Enter your choice: 5
Enter number of rows (1-26): 4
A
A B
A B C
A B C D

```

Result / Conclusion

The menu-driven C program was successfully written, compiled, and executed. All five increasing patterns (rightangled triangle, row-number triangle, natural-number pattern, consecutive-number triangle, and alphabet pattern) were generated correctly using nested loops and separate parameterless functions, without using arrays. The program was tested against the prepared test cases for each activity, and the actual output matched the expected output in every case.